

High-Performance Implementation of Multigrid Solver for Lattice QCD

面向传播子求解的 Clover—Wilson 多重网格实现

CUDA-C++ 后端 · Strict QUDA-style MG · 66 个精确对照单元

张鑫

中国科学院近代物理研究所

2026-10-11 15:05—15:20

汇报人: 张鑫

数据基线: `git tag test27`

项目与证据

GitHub: github.com/zhangxin8069/PyQCU				
Gitee: gitee.com/zhangxin8069/PyQCU				
Docs: .../PyQCU/tree/main/docs				
Tags: .../PyQCU/tags				

Wilson 与 Clover: 先固定可持续求解的细网格算子

同一源码约定下定义质量项、hopping 核与 onsite Clover 块。

Wilson 作用量和 Dslash

$$(H\psi)_x = \sum_{\mu} [(1 - \gamma_{\mu})U_{x,\mu}\psi_{x+\hat{\mu}} + (1 + \gamma_{\mu})U_{x-\hat{\mu},\mu}^{\dagger}\psi_{x-\hat{\mu}}]$$

$$S_W = \sum_x \bar{\psi}_x [(m_0 + 4) - \frac{1}{2}H]\psi_x$$

$$D_W = (m_0 + 4)(I - \kappa H)$$

$$\kappa = \frac{1}{2m_0 + 8}, \quad m_0 + 4 = \frac{1}{2\kappa}$$

PyQCU 求解器使用归一化形式:

$$D_{pc} = I - (\kappa/u_0)H, \quad u_0 = 1$$

C++ `applyWilsonDslashQcu` 返回裸核 H ; 无单位项、无 $-\kappa$ 。

Clover 改进与 onsite 块

$$P_{\mu\nu} = U_{\mu}U_{\nu}U_{\mu}^{\dagger}U_{\nu}^{\dagger}$$

$$C_{\mu\nu} = Q_{\mu\nu} - Q_{\mu\nu}^{\dagger}$$

$$T_p = -\frac{\kappa c_{sw}}{8u_0} \sum_{\mu < \nu} \gamma_{\mu}\gamma_{\nu}C_{\mu\nu}$$

$$A_p = I_p + T_p$$

$$D_{pc}^C = I - (\kappa/u_0)H + T$$

$$D_{phys}^C = (m_0 + 4)I - \frac{1}{2}H - \frac{c_{sw}}{16u_0} \sum_{\mu < \nu} \gamma_{\mu}\gamma_{\nu}C_{\mu\nu}$$

当前 C++ 生产路径: $u_0=1$, 有效 $c_{sw}=1$ 。

$c_{sw} \rightarrow 0$ 时 Clover 路径精确退化为 Wilson。

原始 Dslash: 八个邻居上的矩阵自由稀疏作用

每个格点读取四个正向与四个反向旋量，并在飞行中完成投影和规范平移。

裸跃迁核 H

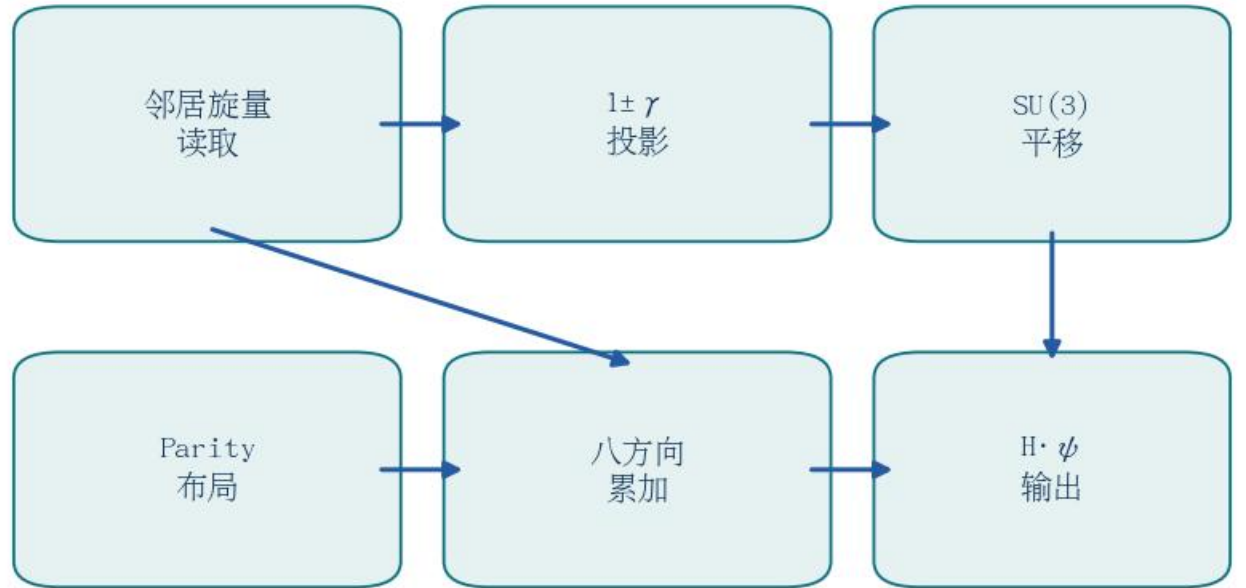
$$(H\psi)_x = \sum_{\mu=0}^3 [(1 - \gamma_\mu) U_{x,\mu} \psi_{x+\hat{\mu}} + (1 + \gamma_\mu) U_{x-\hat{\mu},\mu}^\dagger \psi_{x-\hat{\mu}}]$$

$$D_W = (m_0 + 4) I - (1/2) H$$

H 本身不是完整算子；入口语义必须显式区分。

入口返回对象

<code>applyWilsonDslashQcu</code>	H
<code>give_wilson_eo/oe</code>	$-(\kappa/u_0) H_{pq}$
<code>give_wilson(with_I=True)</code>	$I - (\kappa/u_0) H$
<code>applyCloverDslashQcu</code>	$A_p^{-1} H_{pq}$



为何是性能核心

最近邻模板天然 memory/bandwidth-bound; 4→2 spin projection、SU(3) 重用和 xyzt 尾轴布局共同决定有效带宽。

奇偶消元：外层 Krylov 维度减半，解仍可精确重建

PyQCU 将 Level-0 求解放在奇格子格 Schur 算子 S_o 上。

块矩阵与 Schur 补

$$D_{pc} = [A_e, -\kappa H_{eo}; -\kappa H_{oe}, A_o]$$

$$S_o = A_o - \kappa^2 H_{oe} A_e^{-1} H_{eo}$$

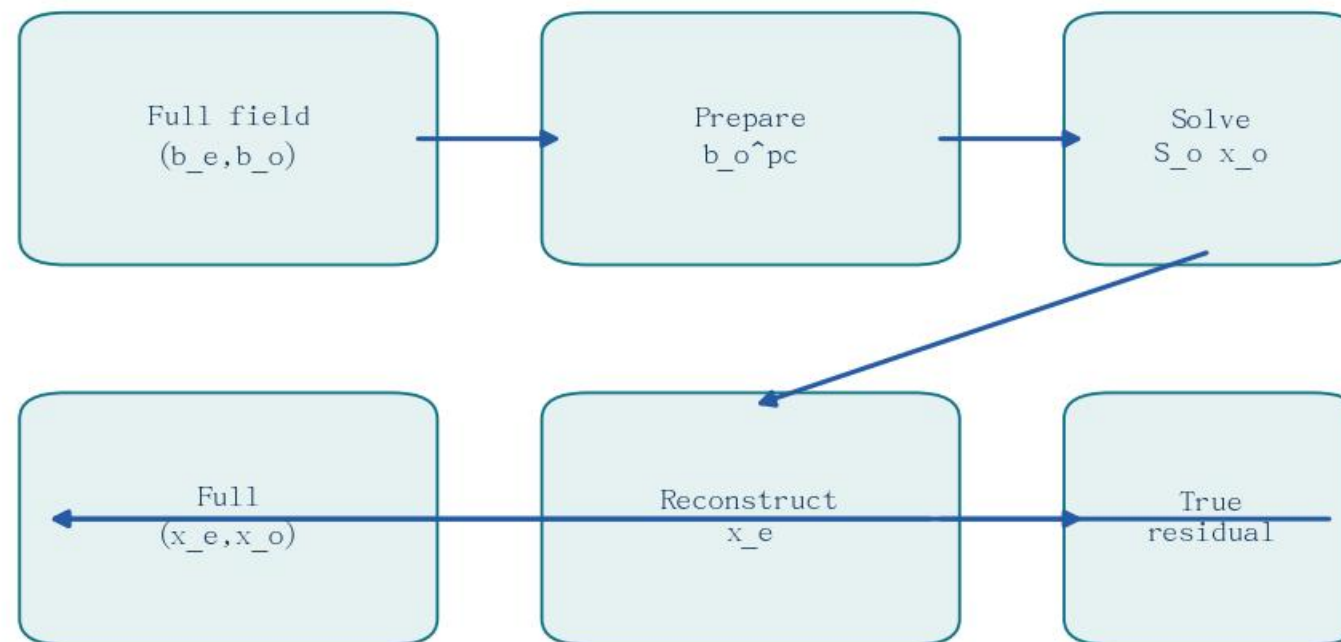
Wilson: $A_e = A_o = I$

求解与重建

$$b_o^{pc} = b_o + \kappa H_{oe} A_e^{-1} b_e$$

$$S_o x_o = b_o^{pc}$$

$$x_e = A_e^{-1} (b_e + \kappa H_{eo} x_o)$$



数值不变量

Schur 求解只改变未知量组织，不改变原方程； D_{ee} 局部块批量求逆，最终使用 full Wilson/Clover 真残差复位。

MultiGrid 的实质：粗空间消除低模，平滑器压低高模

自适应聚集不是“另做一次求解”，而是构造一个小得多的 Galerkin 问题。

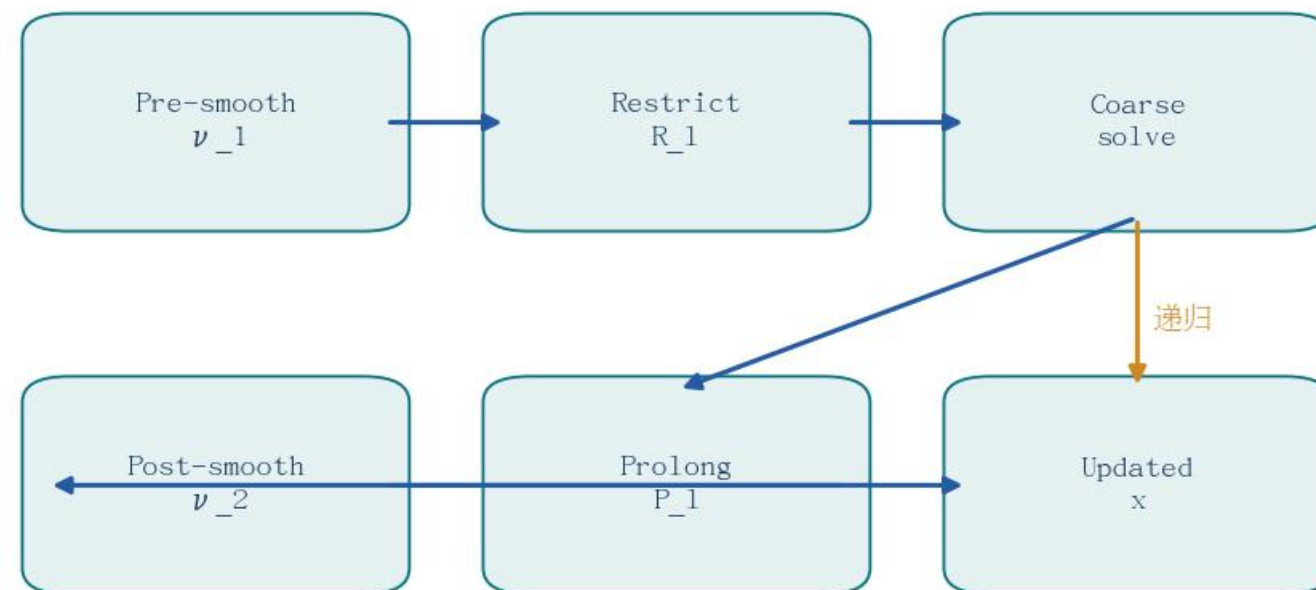
四件数值资产

■ 近零模 B_1 : $A_1 B_1 \approx 0$, 逼近低频子空间。

■ 局部聚集与 QR: $V_1^H V_1 = I$, 块内正交。

■ 转移对: P_1 延拓, $R_1 = P_1^H$ 限制。

$$A_{l+1} = R_l A_l P_l, \quad R_l = P_l^\dagger$$



V-cycle 作为预条件器；外层 Krylov 负责全局收敛。

局部正交为何关键

aggregate 支撑互不重叠，因此 R 不需要全局 Gram 求逆；P 的局部块投影能保留规范背景和 spin-color 耦合。

收敛图像

平滑器处理高频误差，粗层代表低模误差；两者互补，目标是让外层迭代次数弱依赖格点体积。

预条件 Bi-CGStab: 粗层小系统与热启动的核心迭代

以 M 为预条件器; 下表给出可直接核对的完整伪代码。

$$M^{-1}Ax = M^{-1}b$$

$$M\hat{p} = p^{(i)}, \quad M\hat{s} = s$$

$$\omega_i = \frac{t^\dagger s}{t^\dagger t}$$

Algorithm 1 Preconditioned Bi-CGStab: solve $M^{-1}Ax = M^{-1}b$

Input : $A, b, x^{(0)}, M, \varepsilon$

$r^{(0)} = b - Ax^{(0)}$

$\tilde{r} = r^{(0)}$ (for example)

for $i = 1, 2, \dots$

$\rho_{i-1} = \tilde{r}^\dagger r^{(i-1)}$

if $\rho_{i-1} = 0$ **then fail**

if $i = 1$ **then**

$p^{(1)} = r^{(0)}$

else

$\beta_{i-1} = \frac{\alpha_{i-1}\rho_{i-1}}{\omega_{i-1}\rho_{i-2}}$

$p^{(i)} = r^{(i-1)} + \beta_{i-1} (p^{(i-1)} - \omega_{i-1}v^{(i-1)})$

end if

 solve $M\hat{p} = p^{(i)}$

$v^{(i)} = A\hat{p}$

$\alpha_i = \frac{\rho_{i-1}}{\tilde{r}^\dagger v^{(i)}}$

$s = r^{(i-1)} - \alpha_i v^{(i)}$

if $\|s\|/\|b\| \leq \varepsilon$ **then**

$x^{(i)} = x^{(i-1)} + \alpha_i \hat{p}$; **stop**

end if

 solve $M\hat{s} = s$

$t = A\hat{s}$

$\omega_i = \frac{t^\dagger s}{t^\dagger t}$

$r^{(i)} = s - \omega_i t$

$x^{(i)} = x^{(i-1)} + \alpha_i \hat{p} + \omega_i \hat{s}$

end for

PyQCU Strict 特化: $D = X + H$, 并把 QCD 约束嵌入每一层

细层是 target-parity compact 问题; 粗层始终保留 full geometry 和 $X/Y/Yhat$ 资产。

层级分解与 Galerkin 投影

$$D_I = X_I + H_I$$

$$\hat{D}_I = X_I^{-1} D_I$$

$$D_{I+1} = R_I \hat{D}_I P_I$$

先做 onsite 左预条件, 再投影; 顺序不可交换。

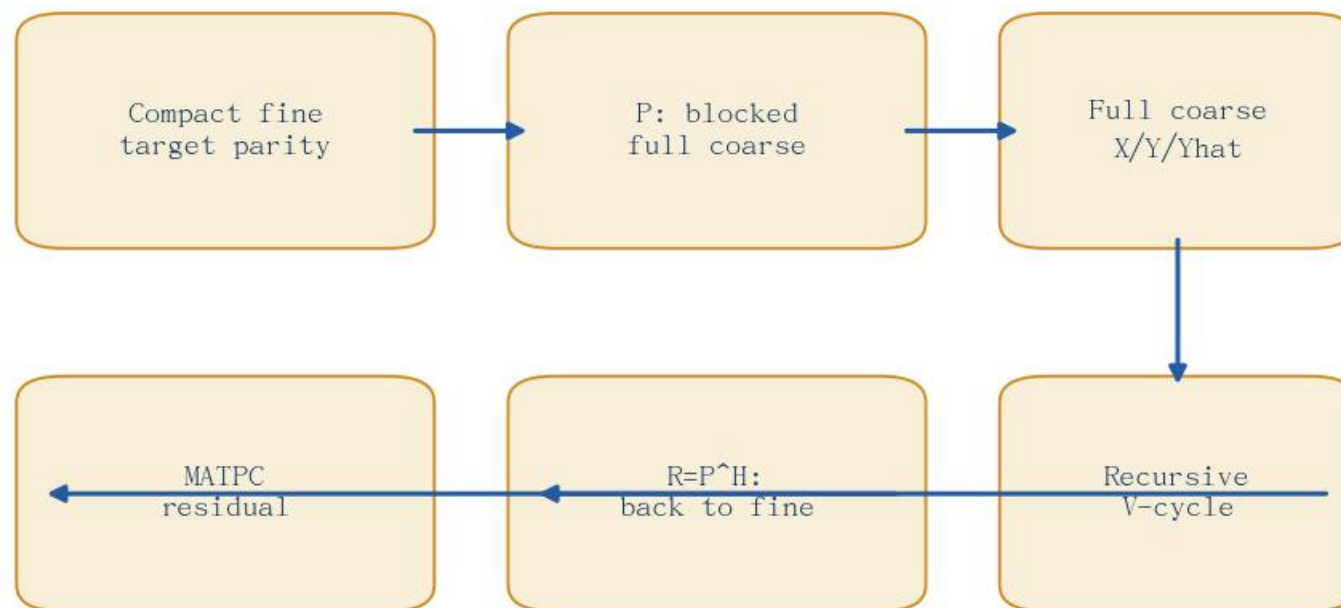
运行资产

X, X^{-1} : onsite / zero-displacement block

Y_f, Y_b : forward / backward coarse links

$$\hat{Y}_f = X^{-1} Y_f, \quad \hat{Y}_b = Y_b X^{-\dagger}$$

raw Y 默认只用于 setup/诊断; 常规 solve 驻留 $Yhat$ 与 X/X^{-1} 。



QCD 的一致性边界

$coarse_spin=2 \cdot nvec$; 粗层不检查板化。P/R 只在 full-coarse 与 compact target-parity fine 之间切换, MATPC 保持原算子语义。

V-cycle 在里、FGMRES 在外: MG 预条件的算法闭环

左表给出递归预条件器, 右表给出 flexible right-preconditioned GMRES。

$$z_l = S_{\text{post}}^{\nu_2}(S_{\text{pre}}^{\nu_1}(r_l) + P_l M_{l+1}^{-1} R_l r_{l+1})$$

Algorithm 2 Recursive MG preconditioner $z_l = \mathcal{M}_l^{-1} r_l$

Input : $A_l, r_l; \{S_{\text{pre}}, S_{\text{post}}, P_l, R_l\}_{\ell=0}^{L-1}$

if $l = L - 1$ **then**

$z_l \leftarrow A_l^{-1} r_l$ (Bi-CGStab)

else

$y_l \leftarrow S_{\text{pre}}^{\nu_1}(r_l)$

$r_{l+1} \leftarrow R_l(r_l - A_l y_l)$

$e_{l+1} \leftarrow \mathcal{M}_{l+1}^{-1} r_{l+1}$

$z_l \leftarrow y_l + P_l e_{l+1}$

$z_l \leftarrow S_{\text{post}}^{\nu_2}(z_l)$

end if

Output : z_l

Update used by the outer solver

$$x^{(k+1)} = x^{(k)} + z^{(k)}, \quad r^{(k+1)} = b - Ax^{(k+1)}.$$

$$z_j = M^{-1} v_j, \quad x \leftarrow x + \sum_j z_j y_j$$

Algorithm 3 Flexible right-preconditioned GMRES

Input : $A, b, x^{(0)}, \mathcal{M}, m, \varepsilon$

$r_0 = b - Ax^{(0)}, \quad \beta = \|r_0\|, \quad v_1 = r_0/\beta$

for $j = 1, \dots, m$

$z_j = \mathcal{M}^{-1} v_j$

$w = Az_j$

for $i = 1, \dots, j$

$h_{ij} = v_i^\dagger w$

$w \leftarrow w - h_{ij} v_i$

end for

$h_{j+1,j} = \|w\|, \quad v_{j+1} = w/h_{j+1,j}$

apply Givens rotations to H_j

if least-squares residual $\leq \varepsilon$ **then break**

end for

$y = \arg \min_y \|\beta e_1 - \bar{H}_j y\|_2$

$x^{(k+1)} = x^{(k)} + Z_j y$

recompute full true residual; restart if needed

$$Z_j = [z_1, \dots, z_j], \quad B_{\text{work}} = (2m + 5)B_f + 2B_c.$$

迭代口径

最细层迭代只记外层 FGMRES; smoother 与 coarse solver 不相加。

工作区

$(2m+5)B_f + 2B_c$; 相同几何与 restart 下跨 solve 复用。

CUDA-C++ 优化从寄存器、流、显存延伸到 MPI

细层解决带宽，粗层解决启动/同步，多 rank 解决 halo；任何单点优化都不足以实用。

细层 kernel

$1\pm r$ 四→二 spin
projection; SU(3) link
重用; xyzt 尾轴布局促发合并访存。

粗层延迟

device_vals 标量; check-
stride; 融合 β/ρ 与 r/x
更新; CUDA Graph 8 迭代段。

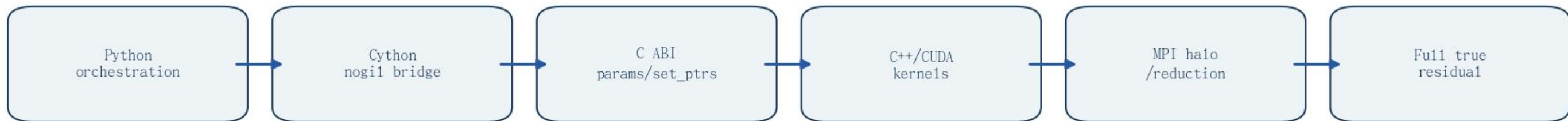
setup / 显存

colored/batched
Galerkin、workspace
预算、持久
hierarchy、schema-v2
SHA256 cache。

通信

rank-local 32 方向粗
halo; face-only 非阻塞
vector
halo; local/remote
分离并融合远端修正。

执行路径



边界: c64 分布式 vector halo 默认 overlap; c128 因浮点累加顺序敏感默认关闭, 仅在显式开关并重验真残差后启用。

对比口径先行：66 个精确单元覆盖设备、精度、层级与 trace

所有正式性能结论来自 git tag test27; trace-on 只用于诊断与阶段归因。

设备	V100 单卡 sm70	双 P100 sm60 / 2×1×1×1
精度	c64	c128
求解	BiCGStab ref	MG-2 / MG-3
trace	off (正式)	on (诊断)
phase	1 cold	2 warmup + 5 steady

请求矩阵与严格完成口径

72	66	132	6
请求 combined	精确 combined	精确 side	容量替代项

统一公平性与验收门

- config hash 66/66; input bundle 66/66。
- 每侧 1 cold + 2 warmup + 5 steady。
- 独立 full-operator 真残差 $< 5 \times 10^{-6}$ 。
- 最大真残差: PyQCU 7.40×10^{-7} , QUDA 7.97×10^{-7} 。
- 速度比定义 $R = t_QUDA / t_PyQCU$; $R > 1$ 表示 PyQCU 更快。

替换不是精确结果

双 P100 c64 16×32×32×48 的 6 个请求项被显式替代为 16^4 ; 替代项只作容量趋势，不进入原大格加速比。

66 个精确单元并非均匀胜出：总中位 1.025，MG 为 35/44

先看全分布和长尾，再决定哪些场景可以宣称优势。

1.025

全部精确单元中位 R

35/44

MG-2/3 有利

21/22

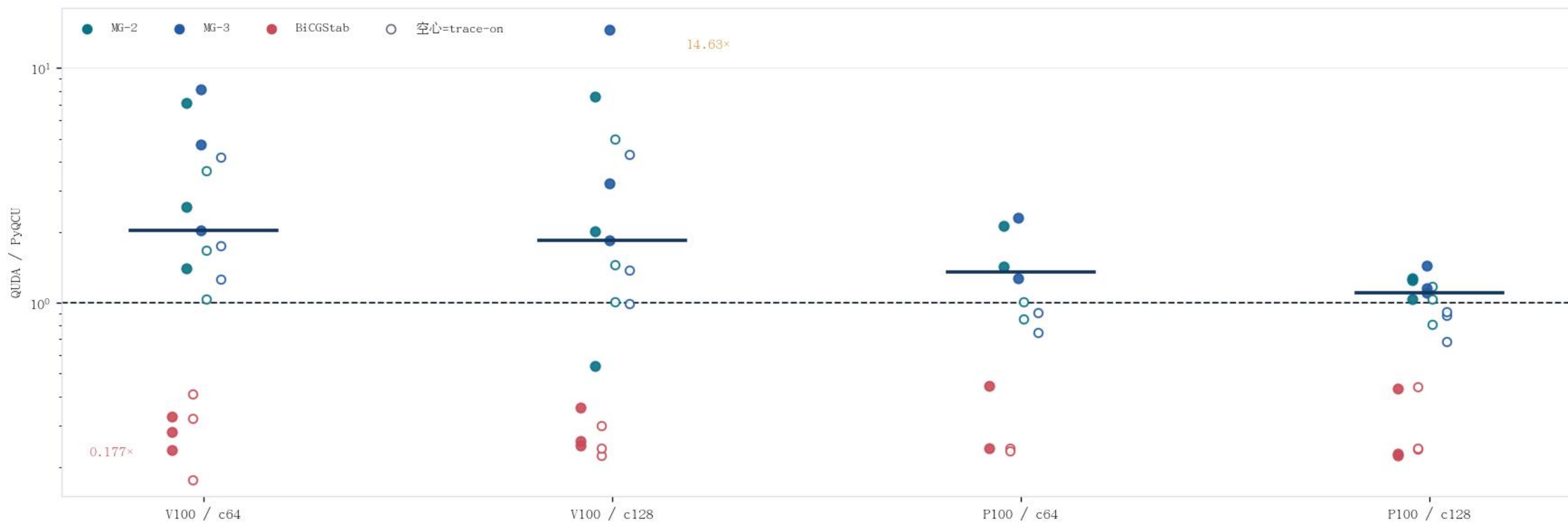
MG trace-off 有利

0/22

BiCGStab 有利

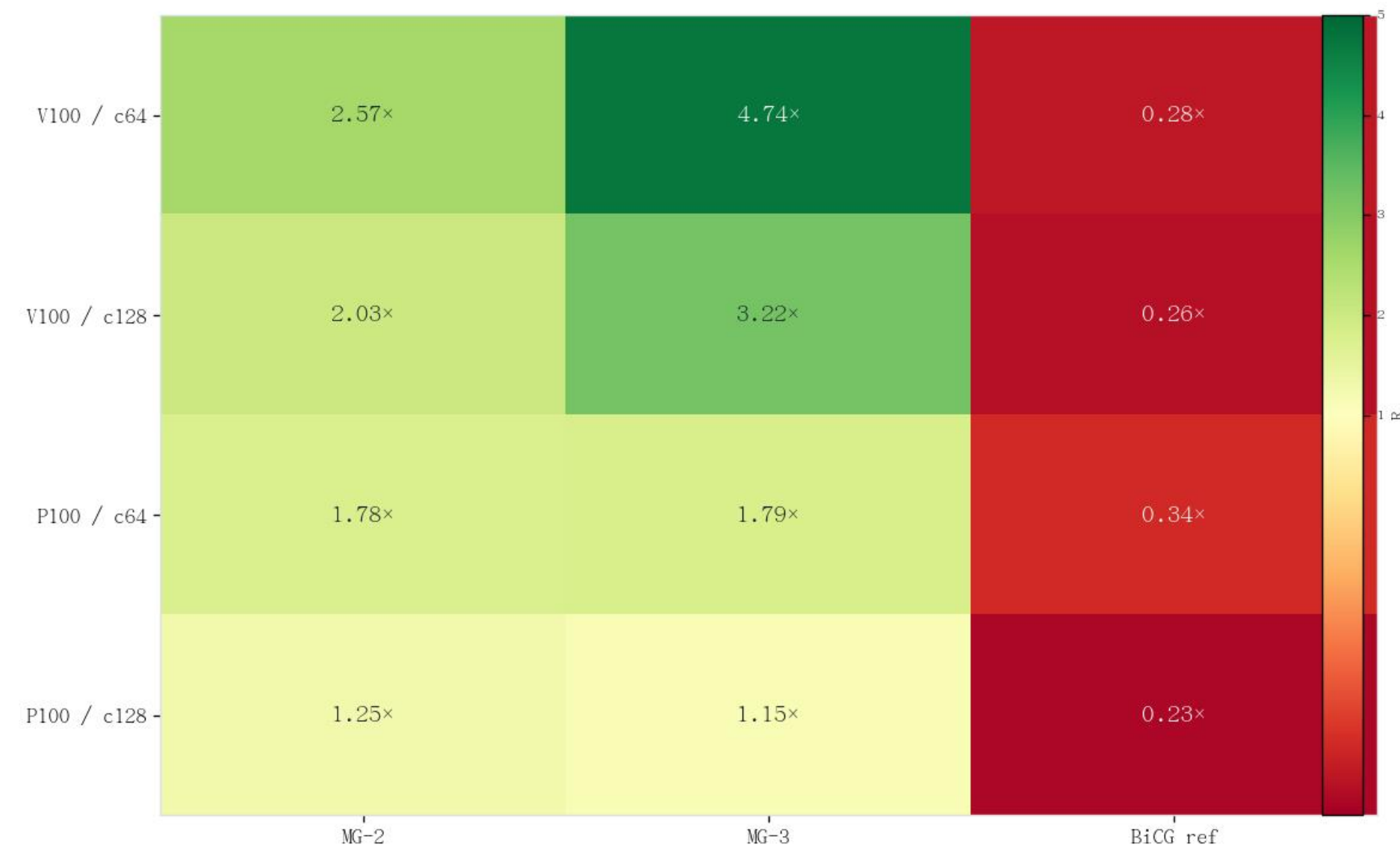
14.6×

最大单点 R



设备与精度决定趋势：V100 明显领先，P100 在 trace-off 总体有利

同一 MG 优势不能外推到 BiCGStab; trace-on 还包含诊断同步开销。



亮点

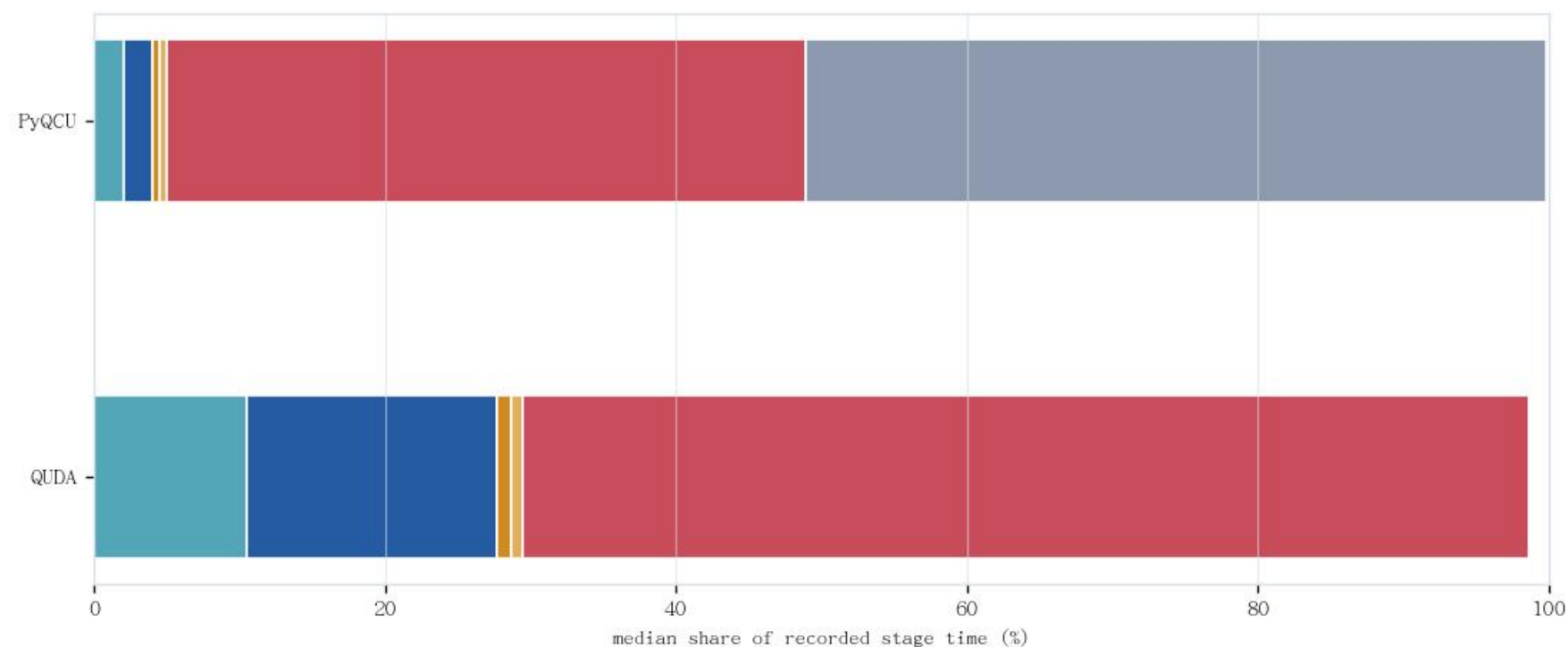
- V100 c64 MG-3: 4.74×
- V100 c128 MG-3: 3.22×
- P100 c64 MG-2/3: 1.78× / 1.79×
- P100 c128 MG-2: 1.25×

必须保留的边界

- MG 总体 35/44 有利，但范围 0.54×-14.63×。
- BiCGStab 0/22, QUDA 明确更快。
- trace-on 部分 P100 组退到 1× 以下。
- P100/c64 每组 n=2, 解释需保守。

阶段与容量证据把优化目标指向粗解、同步和 setup

trace-on 仅用于归因；容量替代必须与原格点性能严格分栏



PyQCU 记录中，coarse solve + other 约占 95%；QUDA 的具名 coarse 约占 69%。

层级同名不代表同一职责，跨库只比较总时间，不叠加同名阶段。

7.40e-7

PyQCU 最大真残差

7.97e-7

QUDA 最大真残差

11,907 MiB

PyQCU 最大观测显存

23,063 MiB

QUDA 最大观测显存

真残差门: 5×10^{-6}

显存为 device-wide sampler；全部保留样本低于 85%。

6 个双 P100 大格请求被替代

请求: 16×32×32×48 c64

默认 overlap 15.9 GB 93.1%

OVERLAP=0 14.8 GB 86.2%

CPU staging 12.4 GB 72.4%

替代为 16^4 ；CPU staging 仍超过 570 s。

当前结论：先兑现 MG 专项优势，再攻克粗层与通信

优势已可验证，但范围、成熟度和容量边界必须和性能数字一起呈现。

优势

扁平单仓库与自主可控；Clover-Wilson 聚焦明确；Strict full-coarse、P/R、X/Y/Yhat 和真残差门完整；trace-off MG 21/22 有利；V100 中位优势显著。

不足

总中位仅 1.025；BiCGStab 0/22；trace-on 同步开销高且方差大；P100 大格 c64 触发容量/时限替代；主要验证拓扑有限，功能覆盖仍偏窄。

下一阶段

通信：持久 request / device-aware；
粗解：减少同步、融合小内核；
setup：流式资产与显存预算；
扩展：作用量、混合精度、国产卡。

可验证的项目主张

$$R = t_{\text{QUDA}}/t_{\text{PyQCU}} > 1 : \text{MG 35/44; trace-off 21/22}$$

不宣称任意格点、任意层数或所有求解器全面领先。

参考实现与复现入口：先跑最小闸门，再进入公平矩阵

PyQCU 的算法来源、对照实现和对内证据均可追溯到源码或 tag。

参考文献与代码

quantum-mg	github.com/weinbe2/quantum-mg Evan Weinberg / NVIDIA
QUDA	github.com/lattice/quda MG / staging / coarse operator
DDalphaAMG	github.com/mrothmann/DDalphaAMG Matthias Rottmann
源码快照	refer/git-rep/** 本地可核验参考树

最短可靠复现接口

```
source ./env.sh

bash ./build.sh && bash ./install.sh

python -B .../run_strict_fast.py --list

python -B .../run_strict_fast.py --fail-fast --json strict-fast.json

mpirun -np 2 python .../strict_mpi_solve_probe.py ...

python -B .../bench_mg_matrix_full.py --list
```

主矩阵与聚合器

bench_mg_matrix_full.py → bench_strict_vs_quda.py →
assemble_final_matrix.py → build_mg_report.py

平台状态

CUDA C++ 已实测；Torch CPU 部分；DCU/CANN 历史或仿真；TileLang
smoke；大规模数据见下一页。

先前成果：508 报告完成 1024 进程规模的大格扩展测试

图表数据取自《张鑫 508应用测试报告-PyQCU》；用于展示早期 DCU 大规模成果，不与 test27 的 MG 速度比混合。

1024

进程/线程规模

3.12×

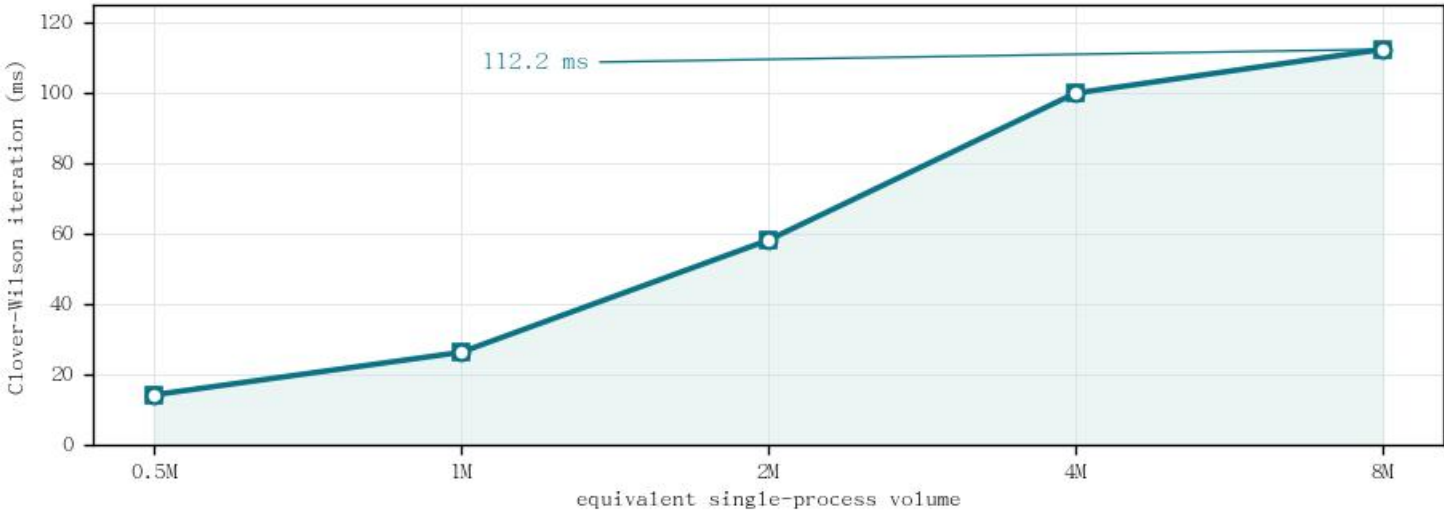
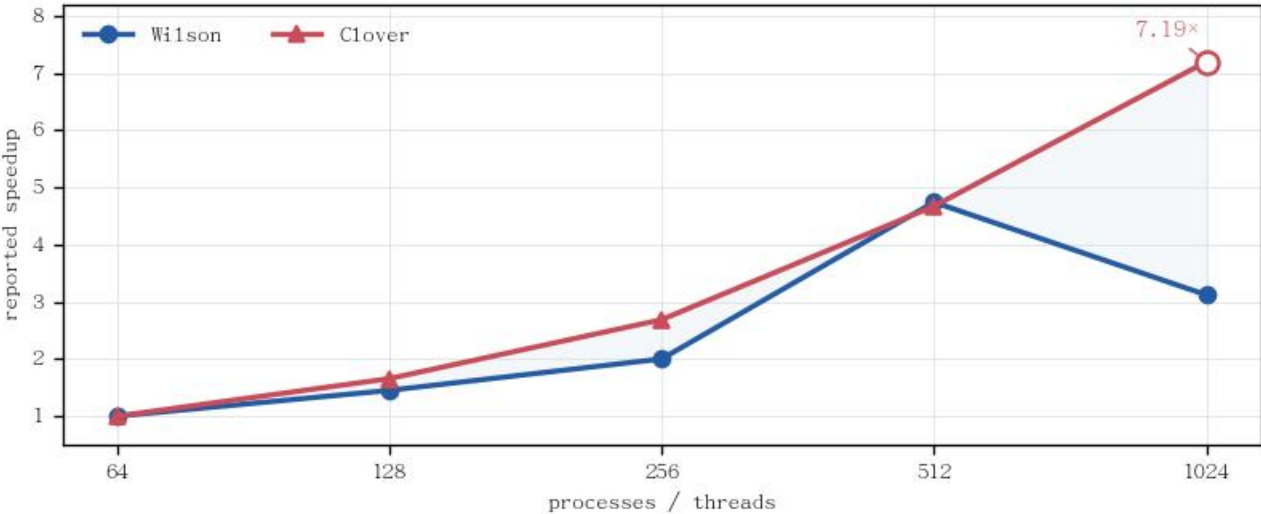
Wilson 报告加速比

7.19×

Clover 报告加速比

≈2048 GiB

64 卡池峰值估算



强扩展：64→1024；数据来自 TEST8/TEST9 汇总。

508 报告原始表：TEST9 强扩展数据

PyQCU：Mass为0.05，tol(x_o)为1e-12，数据类型为complex128，_BLOCK_SIZE_=256，使用节点绑定脚本，使用Wilson与Clover算子时，用面源求解传播子的DCU性能测试，其中 (X:128,Y:128,Z:128,T:256) （具体数据参考<https://gitee.com/zhangxin8069/PyQCU/tree/test9>）

序号	线程数	切分方式	等效单卡格点数量	WilsonBis tabCg迭代数	WilsonBis tabCg最 终残差范 数的平方 (x_e)	WilsonBis tabCg耗 时/s	WilsonBis tabCg单 次迭代耗 时/ms	WilsonBis tabCg加 速比	CloverBis tabCg迭 代数	CloverBis tabCg最 终残差范 数的平方 (x_e)	CloverBis tabCg耗 时/s	CloverBis tabCg单 次迭代耗 时/ms	CloverBis tabCg加 速比
1	128	2,4,4,4	4194304	188	4.306602e-23	0.878154613	88.837398188	1.000	178	2.144e-23	0.85888921	168.2284243	1.000
2	256	4,4,4,4	2097152	156	9.400018e-23	0.953258895	64.5721697	1.257	159	0.340e-23	0.85888921	168.2284243	1.639
3	512	4,4,4,8	1048576	137	3.172852e-23	5.462116316	38.76855375	1.611	137	5.332138e-23	0.882886548	61.83888348	2.756
4	1024	4,4,8,8	524288	157	1.148888e-23	3.979101492	25.34516874	2.211	140	5.795396e-23	0.88718993	59.51888867	4.258

显存峰值占用大概为64GiB+64卡*50%*2=4096GiB

弱扩展差异：Clover-Wilson 单次迭代耗时随等效体积增长。

如何解读

该成果证明早期 DCU/CUDA 路径在多进程大格测试中具备正确性与扩展趋势；但 MG 当时仍属前期验证，不能替代 test27 的 Strict-MG 正式对照。

谢谢

PyQCU Strict MultiGrid

下一步：把粗层同步、通信重叠和 `setup` 显存做成可复验的持续收益。

项目：github.com/zhangxin8069/PyQCU

数据基线：`git tag test27` · 2026-09-28

Q & A